AscendNPU-IR optimization research

Internal work in progress | 19 October 2026

Current H01 and H02 notes for internal discussion. H01 records the objective; H02 remains a partial description of current practices. The attached presentation is an older, unreviewed AI-generated draft.

H01 What are we trying to do

Develop a framework for finding missed optimizations in **AscendNPU-IR**. Produce reproducible reports containing the original code, a proposed replacement, the conditions under which it applies, correctness-check results, and a comparison using the selected improvement metric.

Perspective	What we want to achieve
Customer value	Make the delivery of kernel improvements predictable and less dependent on individual optimization investigation
Developer value	Shorten the investigation and validation work between noticing an opportunity and having an improvement ready for integration.
Technical opportunities	Enable systematic, repeatable experiments that test whether the compiler could produce better code and establish the conditions under which improvements apply

Note: The intended customer benefit is less waiting for useful improvements because finding them consumes scarce expert time.

H02 How is it done today

Working notes based on the project owner's observations. This is a partial account of the **AscendNPU-IR** team's workflow; the uncertainties below still need investigation.

Current practices

- **Performance regression testing:** nightly testing aims to reduce the number of performance regressions that reach upstream. Reverting code changes is one response to detected regressions.
- **Performance issue tickets:** newly discovered performance issues in already merged code are assigned to developers for investigation.
- **Kernel-specific tasks:** some tasks ask engineers to make a particular kernel faster without prescribing an optimization. Investigation includes inspecting the generated code.
- **Optimization epics:** broader tasks target improvements to a class of optimizations. Comparisons with other accelerators are a likely source of these epics, although this has not been confirmed.

Q02: How are kernel-specific optimization tasks and optimization epics selected and prioritized?

Identifying slow cases

Comparing the same kernel across GPU/NPU platforms can reveal relatively slow cases and motivate optimization work. The working hypothesis is that comparisons against CUDA implementations motivate some kernel-specific tasks and optimization epics; their actual role in the team's workflow needs confirmation.

A performance gap provides a reason to investigate. It does not, by itself, establish a missed compiler optimization or show how much of the gap a compiler change could eliminate.

A possible implication for the proposal is included under Supporting material below. This remains a hypothesis to investigate.

Nightly regression testing provides another signal: performance has deteriorated relative to an existing baseline. A stable baseline can still contain missed optimization opportunities.

Q03: What proactive searches for missed optimizations already take place, and how are they organized?

Related terminology research: Q01 - Taxonomy of bug finding and performance search.

Investigating an optimization task

A perf-like profiling tool is available, but its name and capabilities need clarification.

The difficulties vary by task. Reported examples include insufficient runnable examples for profiling, complex IR that is hard to inspect, uncertainty about a suitable optimization, and validation tools that require substantial manual reasoning. Occasionally, identifying where in the compiler to implement a fix is also difficult.

These observations describe the working environment. They do not establish a commitment for this project to solve those difficulties.

Q04: What tools or processes already help engineers turn a slow case into a candidate compiler improvement?

Supporting material

Framework generated comparison references

Status: hypothesis from discussion; not validated.

If kernel-specific tasks and optimization epics are often prompted by comparisons with other accelerators, the framework could offer another way to identify opportunities: generate an alternative for the same workload and compare it with the existing code on the same Ascend NPU target.

The reference would be generated automatically by the framework. A candidate that passes the agreed correctness checks and improves the selected metric could provide evidence of an optimization opportunity without requiring an implementation for another accelerator as the reference.

This may be an advantage of the approach. It depends on both the actual role of cross-platform comparisons in the current workflow and the framework's ability to generate useful alternatives. It is not yet an established benefit or a claim of novelty.

Papers informing the work

- **LPO: Discovering Missed Peephole Optimizations with Large Language Models.** arXiv:2508.16125v2, 16 March 2026. Local PDF: `papers/LPO-2508.16125v2.pdf`. Versioned paper record.
- **PRAGMA: A Profiling-Reasoned Multi-Agent Framework for Automatic Kernel Optimization.** arXiv:2511.06345v2, 24 November 2025. Local PDF: `papers/PRAGMA-2511.06345v2.pdf`. Versioned paper record.

Earlier AI generated presentation

The existing 25-slide presentation is included unchanged in `presentation/UNREVIEWED-AI-DRAFT.pptx` and `presentation/UNREVIEWED-AI-DRAFT.pdf`. It predates the rebuilt vault and contains assumptions that have not been adopted as project decisions. Use it as background while reading the current notes above.